

Report - Version 2: Multiclass Neural Network

Dataset	Task	Train acc.	Test acc.
Fashion-MNIST	10 classes	83.13%	83.42%

Introduction

Version 2 extends the first binary model to the complete 10-class Fashion-MNIST problem. The goal is to preserve the same transparent learning loop while replacing one sigmoid output with 10 class scores and softmax probabilities.

The model is still intentionally simple: there are no hidden layers. Each image is mapped directly from 784 normalized pixel values to 10 output scores.

Goal of Version 2

The main goal is to understand multiclass classification: how a model represents 10 classes, converts scores into probabilities, measures multiclass error, and updates a matrix of weights and a vector of biases.

Data and Preparation

The project uses the Fashion-MNIST CSV dataset. The training file contains 60,000 labeled images and the separate test file contains 10,000 labeled images. Each 28 x 28 grayscale image is represented by 784 pixel values.

All 10 classes are used. Pixel values are normalized from 0-255 to 0-1 by dividing by 255. The integer labels are also converted to one-hot vectors for multiclass cross-entropy.

Architecture

The flow of Version 2 is:

784 pixels -> 10 linear scores -> softmax -> class probabilities -> argmax prediction

Multiclass Output

Weights now form a 784 x 10 matrix and the bias contains 10 values. For every image, the linear step produces 10 raw scores, one for each Fashion-MNIST class.

$$z = X @ W + b$$

Softmax Activation

Softmax converts the 10 raw scores into 10 probabilities that sum to 1. The predicted class is the index of the highest probability.

Cross-Entropy and Gradient Descent

Multiclass cross-entropy measures how much probability the model assigns to the correct class. Gradient descent then adjusts all weights and biases in the direction that reduces this loss.

The learning rate is 0.1. Training stops when the improvement in loss becomes smaller than $\text{tol} = 1\text{e-}4$. This tolerance is a practical compromise: a smaller value can slightly improve accuracy but requires more full-batch iterations and therefore more execution time.

Training Result

With this simple linear architecture, the final training loss is approximately 0.5092 and the training accuracy is 83.13%.

Evaluation

The separate Fashion-MNIST test CSV is normalized in the same way. Softmax probabilities are computed and `argmax` selects one class from 0 to 9. The resulting test accuracy is 83.42%.

Results

Metric	Value
Training loss	0.5092
Training accuracy	83.13%
Test accuracy	83.42%

What Version 2 Teaches

Version 2 shows how the binary learning loop generalizes to many classes without introducing hidden layers. A single linear layer can already learn useful class boundaries, but the model is limited to relationships that can be represented by this direct mapping from pixels to outputs.

It also introduces the core multiclass components used by larger neural networks: one-hot targets, softmax probabilities, multiclass cross-entropy, a weight matrix, and argmax prediction.

Accuracy and Execution-Time Trade-off

The stopping tolerance $\text{tol} = 1\text{e-}4$ keeps this educational implementation reasonably fast. Lowering the tolerance makes training continue for more iterations and can improve the result slightly, but it also increases execution time. This trade-off will be addressed naturally as the architecture evolves.

Limitations

Version 2 has no hidden layers, uses full-batch gradient descent, and learns only a linear mapping from the 784 input pixels to the 10 classes. It therefore cannot yet learn richer non-linear representations of the images.

Conclusion

Version 2 successfully extends the project from a two-class single-neuron model to a complete 10-class Fashion-MNIST classifier. The model reaches 83.13% training accuracy and 83.42% test accuracy while keeping the implementation compact and fully transparent.

The next natural evolution is to increase the representational power of the network while preserving the same from-scratch learning logic.